

Assignment #3: Stack, DP & Backtracking

Updated 2226 GMT+8 Sep 22, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

1078: Bigram分词

<https://leetcode.cn/problems/occurrences-after-bigram/>

思路：

解题思路

将文本按空格分割成单词列表

遍历单词列表，检查连续的两个词

如果前两个词分别匹配 first 和 second，则将第三个词加入结果

返回结果列表

代码：

```
class Solution:
    def findOccurrences(self, text, first, second):
        words = text.split()
        result = []

        for i in range(len(words) - 2):
            if words[i] == first and words[i + 1] == second:
                result.append(words[i + 2])

        return result
```

代码运行截图

通过

30 / 30 个通过的测试用例

官方题解

写题解

Youthful Spencezwr

提交于 2025.10.08 17:44



面向在校生的优惠升级方案

完成认证享 1 元/天升级 Plus 会员，享更多学业及职业成长帮助

→

🕒 执行用时分布

0 ms | 击败 100.00% 🏆

📊 复杂度分析

💡 消耗内存分布

12.29 MB | 击败 35.71%

283.移动零

stack, two pointers, <https://leetcode.cn/problems/move-zeroes/>

思路：

当遇到非零元素时，直接与左指针位置交换，避免后续填充0的操作

代码：

```
class Solution(object):
    def moveZeroes(self, nums):

        left = 0

        for right in range(len(nums)):
            if nums[right] != 0:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1
```

代码运行截图

通过 75 / 75 个通过的测试用例

Youthful Spencezwr 提交于 2025.10.08 17:37

官方题解

写题解



面向在校生的优惠升级方案

完成认证享 1 元/天升级 Plus 会员，享更多学业及职业成长帮助

→

🕒 执行用时分布

3 ms | 击败 89.54% 🌿

📊 复杂度分析

💾 消耗内存分布

13.31 MB | 击败 50.72% 🌿

📊 复杂度分析

20.有效的括号

stack, <https://leetcode.cn/problems/valid-parentheses/>

思路：

使用栈来解决：

遇到左括号：压入栈中

遇到右括号：检查栈顶是否为对应的左括号

如果匹配，弹出栈顶

如果不匹配或栈为空，返回 false

遍历结束：栈为空则有效，否则无效

代码：

```
class Solution:
    def isValid(self, s):
        stack = []
        mapping = {'(': ')', ')': '(', '[': ']', ']': '['}

        for char in s:
            if char in mapping:
                top_element = stack.pop() if stack else '#'
                if mapping[char] != top_element:
                    return False
            else:
                stack.append(char)

        return len(stack) == 0
```

代码运行截图

通过

102 / 102 个通过的测试用例

Youthful Spencezwr 提交于 2025.10.08 17:49

官方题解

写题解



尊享面试 100 题
专为会员定制面试题单，涵盖完整知识架构与更多会员面试真题，精心布局刷题顺序查漏补缺。

→

⌚ 执行用时分布

4 ms | 击败 40.34%

🔮 复杂度分析

💾 消耗内存分布

12.46 MB | 击败 27.81%

118.杨辉三角

dp, <https://leetcode.cn/problems/pascals-triangle/>

思路:

每一行的第一个和最后一个元素都是 1

中间的元素等于上一行的相邻两个元素之和

第 i 行有 i 个元素(从1开始计数)

代码:

```
class Solution(object):
    def generate(self, numRows):
        if numRows == 0:
            return []
        result = []

        for i in range(numRows):
            row = [1] * (i + 1)
            for j in range(1, i):
                row[j] = result[i-1][j-1] + result[i-1][j]

            result.append(row)

        return result
```

代码运行截图

通过 30 / 30 个通过的测试用例

Youthful Spencezwr 提交于 2025.10.08 17:56

官方题解

写题解

 **尊享面试 100 题**
专为会员定制面试题单，涵盖完整知识架构与更多会员面试真题，精心布局刷题顺序查漏补缺。 →

46.全排列

backtracking, <https://leetcode.cn/problems/permutations/>

思路：

维护一个 path 数组存储当前排列

用 visited 数组标记已使用的数字

每次从所有未使用的数字中选一个加入路径

递归构建,完成后回溯

代码

```

class Solution(object):
    def permute(self, nums):

        result = []
        path = []
        visited = [False] * len(nums)

        def backtrack():
            if len(path) == len(nums):
                result.append(path[:])
                return

            for i in range(len(nums)):

                if visited[i]:
                    continue

                path.append(nums[i])
                visited[i] = True

                backtrack()

                path.pop()
                visited[i] = False

        backtrack()
        return result

```

通过 26 / 26 个通过的测试用例

官方题解

写题解

Youthful Spencezwr 提交于 2025.10.08 18:03

🕒 执行用时分布

📖

7 ms | 击败 11.80%

🌟 复杂度分析

💾 消耗内存分布

12.42 MB | 击败 70.15% 🌱

40%

78.子集

backtracking, <https://leetcode.cn/problems/subsets/>

思路：

这道题可以用回溯法（DFS）来解决。核心思想是：对于每个元素，我们都有两种选择：

- 1.选择它，加入当前子集
- 2.不选择它，跳过

通过递归遍历所有可能性，就能得到所有子集。

代码

```
class Solution(object):
    def subsets(self, nums):
        result = []
        path = []
        def backtrack(start):
            result.append(path[:])
            for i in range(start, len(nums)):
                path.append(nums[i])
                backtrack(i + 1)
                path.pop()

        backtrack(0)
        return result
```

通过

10 / 10 个通过的测试用例

Youthful Spencezwr

提交于 2025.10.08 18:07

官方题解

写题解

🕒 执行用时分布

0 ms | 击败 100.00% 🏆

📊 复杂度分析

💾 消耗内存分布

12.50 MB | 击败 38.26%

📊 复杂度分析

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

通过本周的作业题，我主要学习掌握了之前在计概学习中只是理解概念而不会应用的递归。